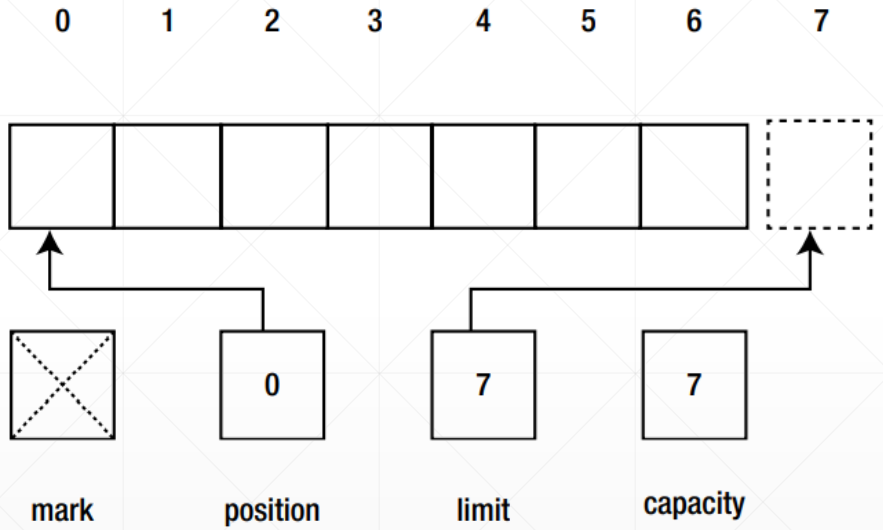
理解Buffer的关键属性

北京理工大学计算机学院
金旭亮

Buffer的四个关键属性

- capacity（容量）
- limit（限制）
- position（位置）
- mark（标记）

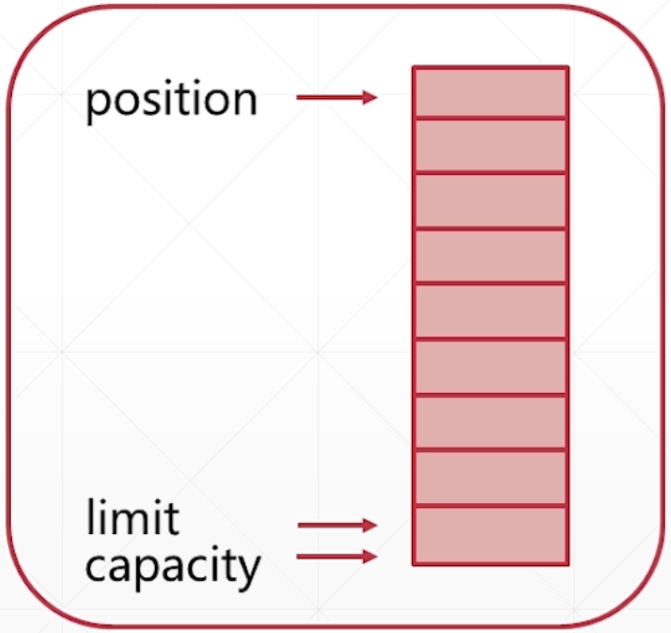
一个包容了7个元素的Buffer示例



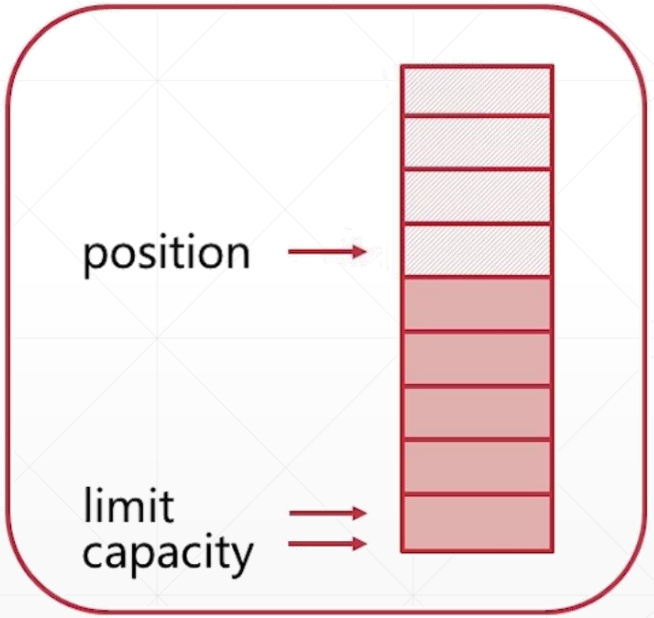
$$0 \leqslant mark \leqslant position \leqslant limit \leqslant capacity$$

理解Buffer的工作原理-1

初始状态



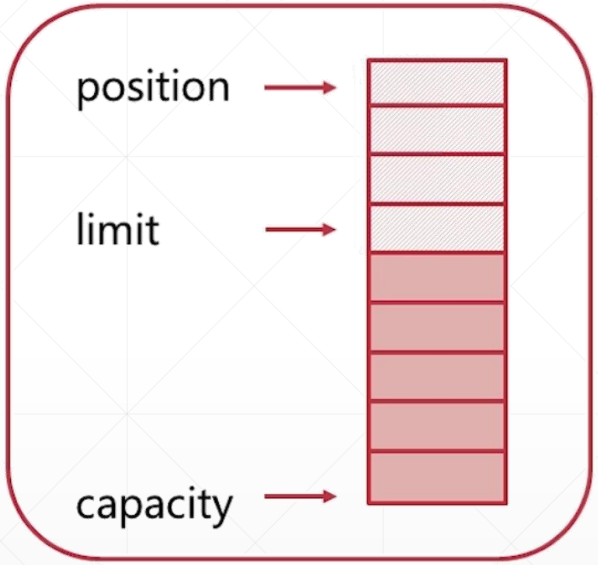
写入四个数据



position代表当前数据读写位置。

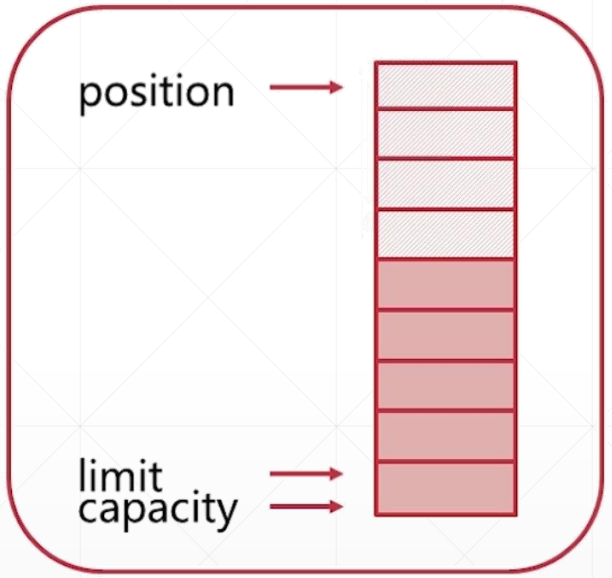
理解Buffer的工作原理-2

flip()



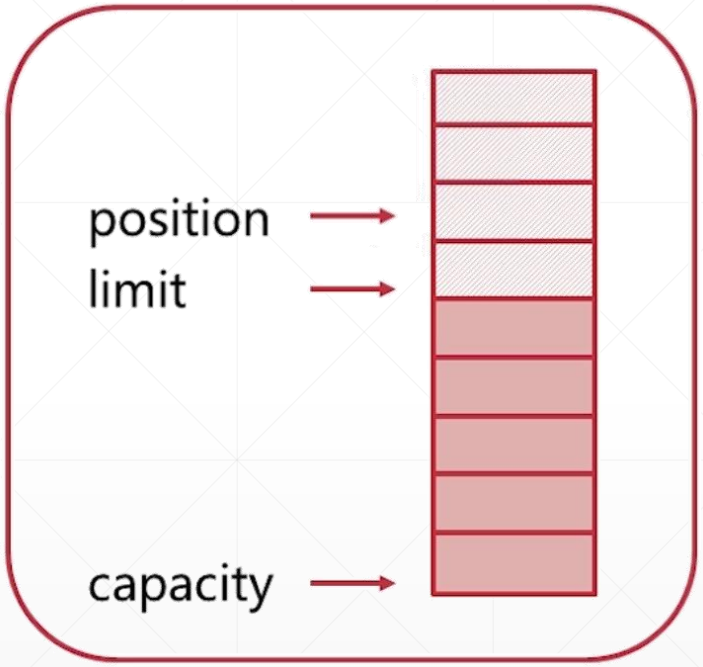
flip()将指针移回开头，limit指向最后写入的数据，这样就可以读出已经写入的数据了。

clear()



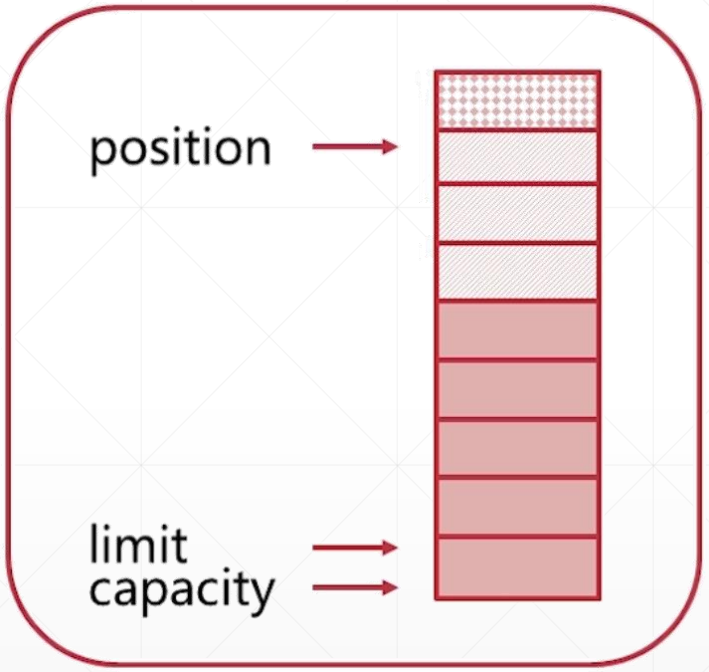
clear()方法将position移到开头，limit回到末尾，但并不清除已经在缓冲区中的数据。

理解Buffer的工作原理-3



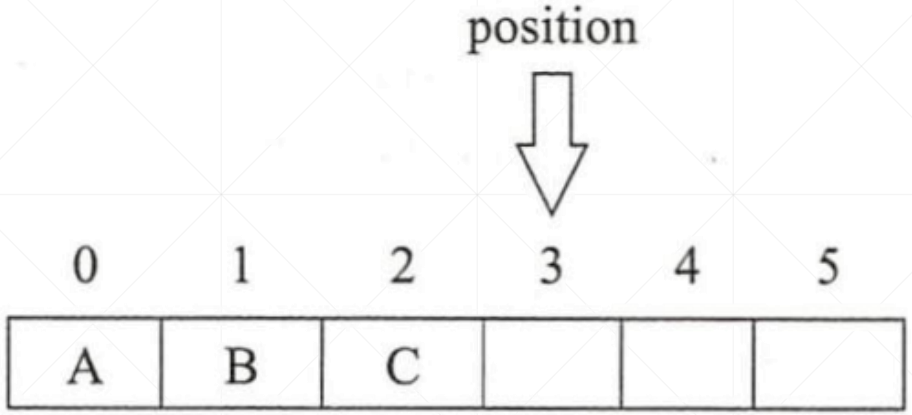
如果仅读了部分数据……

compact()



compact()操作将导致已读数据被移除，未读数据保留在前部，limit移到末尾。

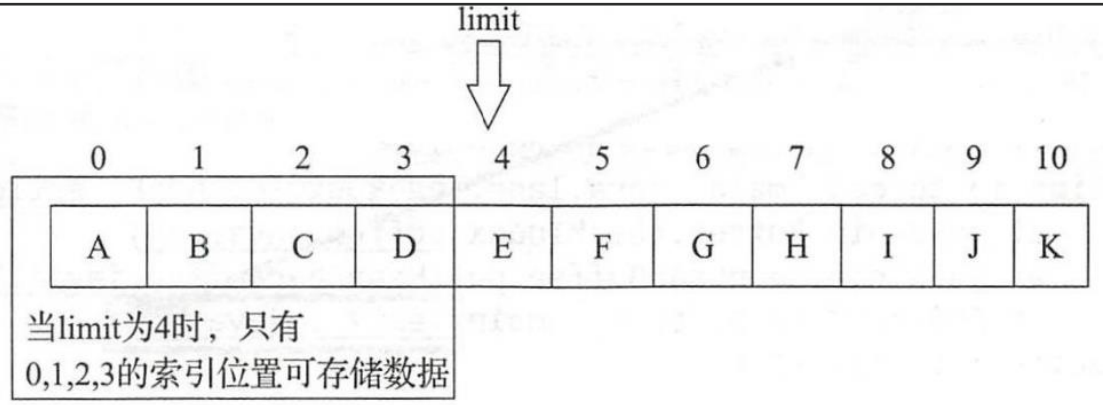
position的作用



position代表“下一个”要读取或写入元素的index（索引），缓冲区的position（位置）不能为负 ，并且position不能大于其limit。

limit的含义

代表第一个不应该读取或写入元素的index（索引）。



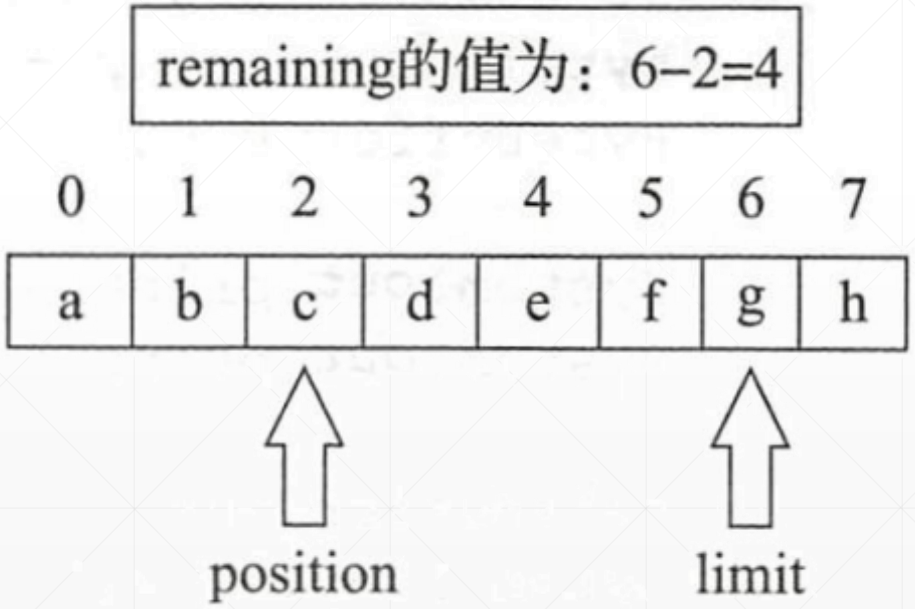
如果position大于新的limit，则将position设置为新的limit。

使用get方法尝试读取超过limit的元素时，会得到IndexOutOfBoundsException。

remaining值的计算

为了清楚地知道到底还有多少个数据可供读取，Buffer提供了一个hasRemaining()方法，只要还有数据可供读取，此方法就返回true。

remaining的计算方法很简单，就是limit-position，只要此值不为0，就意味着还有数据可供读取。



示例

```java
public class LimitAndPosition {
    public static void main(String[] args) {
        Buffer buffer = ByteBuffer.allocate( capacity: 7);
        MyBufferUtils.printBufferInfo(buffer);

        System.out.println("Changing buffer limit to 5");
        buffer.limit( newLimit: 5);
        MyBufferUtils.printBufferInfo(buffer);
        System.out.println("Changing buffer position to 3");
        buffer.position( newPosition: 3);
        MyBufferUtils.printBufferInfo(buffer);
        //java.nio.HeapByteBuffer[pos=3 lim=5 cap=7]
        System.out.println(buffer);
    }
}
```

```
================================
Capacity: 7
Limit: 7
Position: 0
Remaining: 7
================================
Changing buffer limit to 5
================================
Capacity: 7
Limit: 5
Position: 0
Remaining: 5
================================
Changing buffer position to 3
================================
Capacity: 7
Limit: 5
Position: 3
Remaining: 2
================================
java.nio.HeapByteBuffer[pos=3 lim=5 cap=7]
```

LimitAndPosition.java

存入数据后各属性值的变化

```java
//展示存入数据后各参数值的变化
public class BufferInfoAfterPutData {
    public static void main(String[] args) {
        ByteBuffer buffer = ByteBuffer.allocate( capacity: 7);
        MyBufferUtils.printBufferInfo(buffer);
        //写入数据
        buffer.put((byte) 10).put((byte) 20).put((byte) 30);
        MyBufferUtils.printBufferInfo(buffer);
        //输出其内容
        for (int i = 0; i < buffer.limit(); i++)
            System.out.print(buffer.get(i) + ",");
    }
}
```

```
Run    BufferInfoAfterPutData
"C:\Program Files\Java\jdk-20\bin\java.exe"
================================
Capacity: 7
Limit: 7
Position: 0
Remaining: 7
================================
================================
Capacity: 7
Limit: 7
Position: 3
Remaining: 4
================================
10,20,30,0,0,0,0,
Process finished with exit code 0
```

理解Limit的作用

```java
public class TestLimit {
    public static void main(String[] args) {
        char[] charArr = {'a', 'b', 'c', 'd', 'e'};
        CharBuffer charBuffer = CharBuffer.wrap(charArr);

        System.out.println(charBuffer.length());//5
        System.out.println(charBuffer.capacity());//5
        System.out.println(charBuffer.limit());//5
        System.out.println(charBuffer.position());//0

        charBuffer.limit( newLimit: 2);
        System.out.println(charBuffer.limit());//5
        System.out.println(charBuffer.get(1));//b
        //超过Limit,抛出IndexOutOfBoundsException
        System.out.println(charBuffer.get(2));
    }
}
```

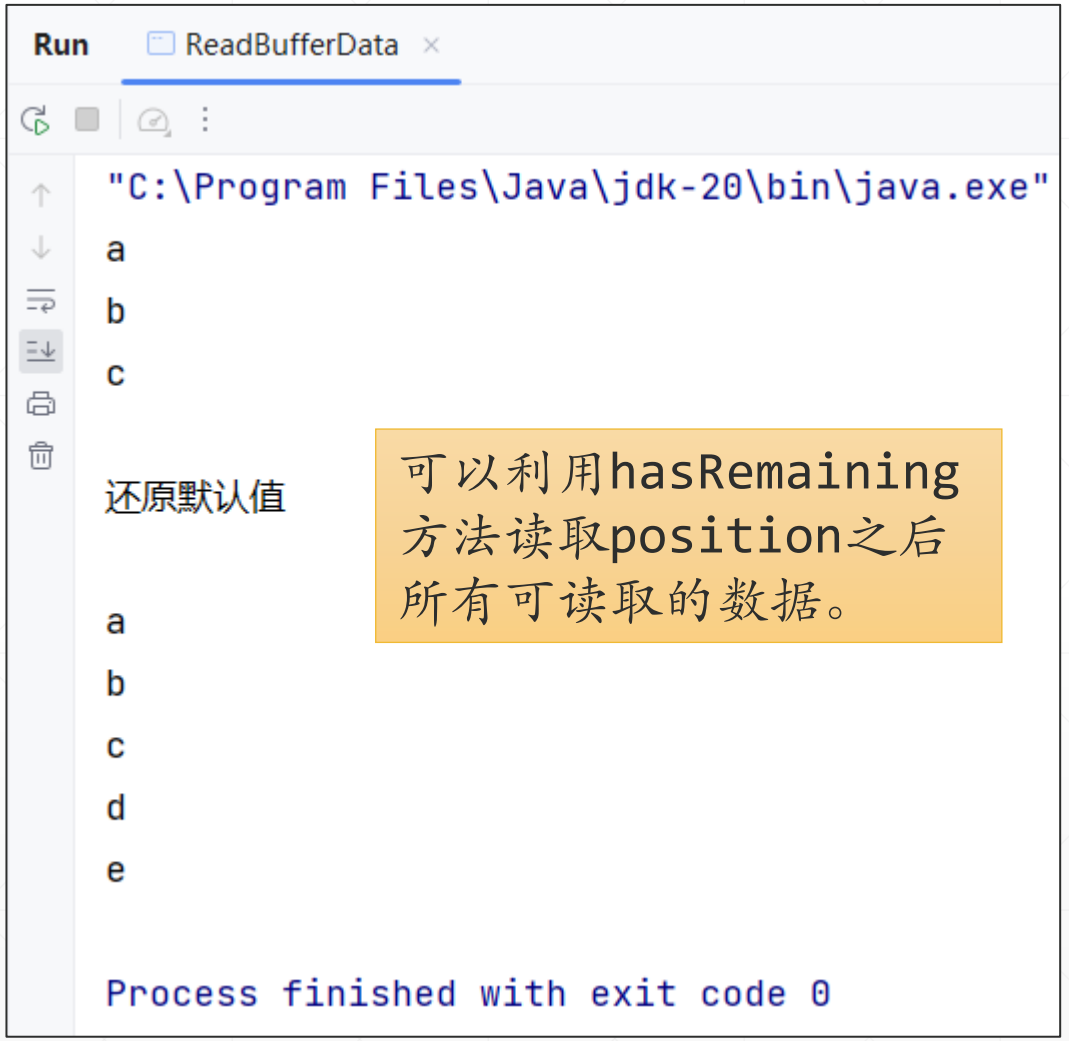
BufferInfoDemo.java

当尝试着存取超过limit的数据时，会抛出异常。

读取所有可读取的数据

```java
private static void readAvailableData() {
    char[] charArr = {'a', 'b', 'c', 'd', 'e'};
    CharBuffer charBuffer = CharBuffer.wrap(charArr);
    charBuffer.limit(3);
    charBuffer.position(0);
    //当还有数据可读时，hasRemaining()=true
    while(charBuffer.hasRemaining()){
        System.out.println(charBuffer.get());
    }
    System.out.println("\n还原默认值\n");
    charBuffer.clear();
    while(charBuffer.hasRemaining()){
        System.out.println(charBuffer.get());
    }
}
```

ReadBufferData.java



可以利用hasRemaining方法读取position之后所有可读取的数据。